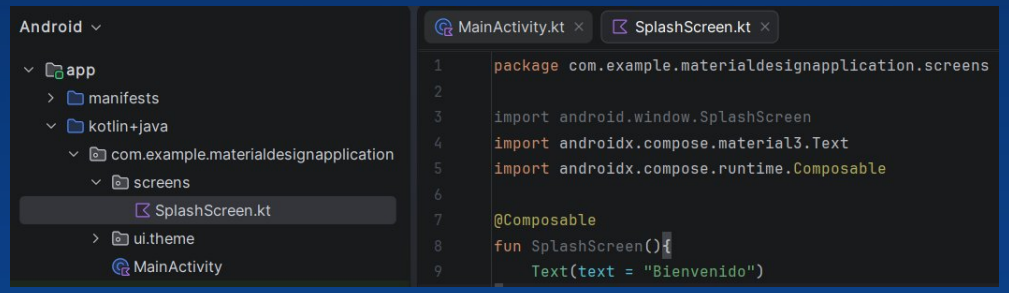
# UD4. Configurar temas con Material Design

# 0. índice

1. Introducción
2. Imagenes
3. Mensajes
4. Fuentes
5. Colores
6. SplashActivity
7. Animaciones

# 1. Introducción

- Seguimos avanzando en el curso de Desarrollo de interfaces. Lo siguiente que vamos a ver es como organizar todos nuestros recursos cuando trabajamos con Material Design.
- Creamos un nuevo proyecto y en la función principal creamos la llamada a una función llamada SplashScreen().
- Creamos un nuevo package llamado screens, donde crearemos una nueva clase kotlin de tipo file llamada SplashScreen.kt.
- Dentro creamos la función SplashScreen que saca un texto de bienvenida.



# 2. Imagenes

- En la función SplashScreen vamos a meter el texto en una columna para alinear elementos verticalmente con verticalArrangement y horizontalAlignment para centrar.

- Por encima del texto, añadiremos una imagen. Buscaremos un JPG, lo copiamos en res —> drawable y usamos painterResource(R.drawable.nombreImagen) para añadir la imagen.

```
13  @Composable
14  fun SplashScreen(){
15      Column (
16          verticalArrangement = Arrangement.Center,
17          horizontalAlignment = Alignment.CenterHorizontally
18      ){
19          Image(
20              painter = painterResource(id = R.drawable.im_grabi_1),
21              contentDescription = "imagen")
22          Text(text = "Bienvenido")
23      }
24
25  }
```

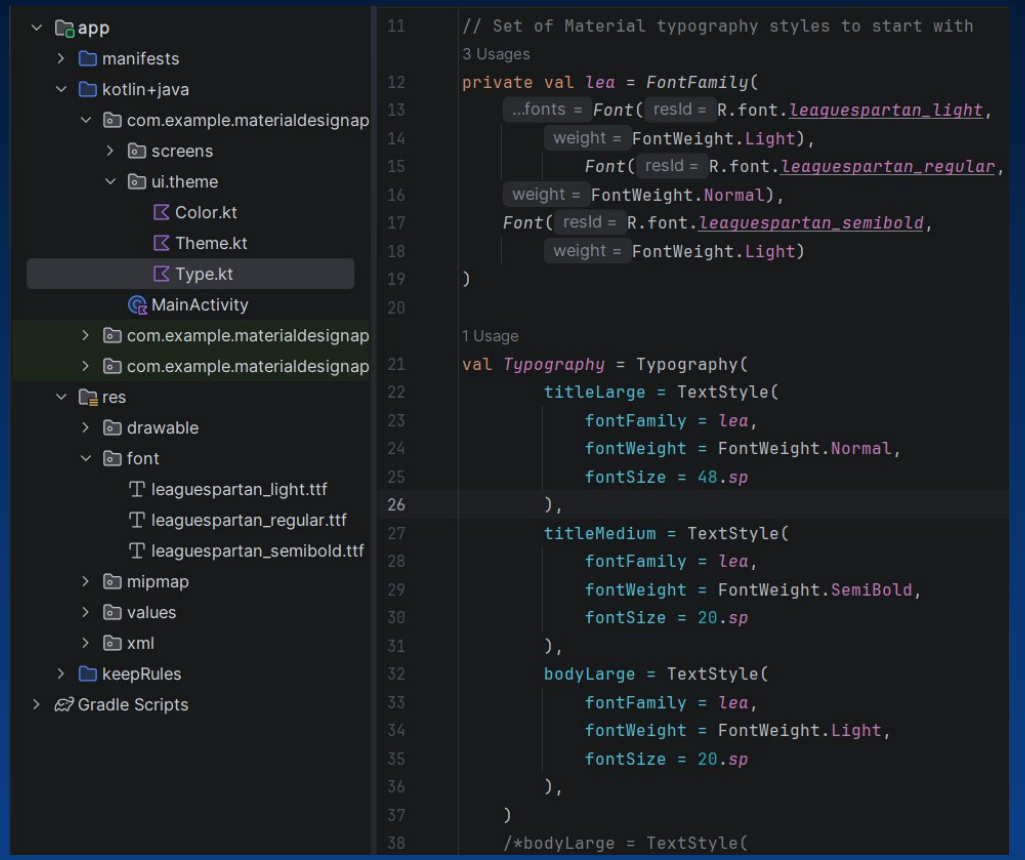
# 3. Mensajes

- Añadimos un botón que escriba un mensaje en consola con Log.d para verificar en Logcat y que contenta el texto “Continuar”.

```
15  @Composable
16  fun SplashScreen(){
17      Column (
18          verticalArrangement = Arrangement.Center,
19          horizontalAlignment = Alignment.CenterHorizontally
20      ){
21          Image(
22              painter = painterResource(id = R.drawable.im_grabi_1),
23              contentDescription = "imagen")
24          Text(text = "Bienvenido")
25          OutlinedButton(onClick = { Log.d( tag = "SplashScreen",  msg = "Haga click en continuar")
26
27          }) {
28              Text(text = "Continuar")
29          }
30      }
31
32  }
```

# 4. Fuentes

- Nos descargamos las fuentes que queramos usar de páginas como https://fonts.google.com/ y podemos previsualizarlas en https://typescale.com/.
- Metemos los archivos .ttf en una carpeta que se llame exactamente “font”.
- Copiamos y pegamos esta carpeta en la carpeta res del proyecto. Las fuentes tienen que estar en minúsculas y con guiones bajos.
- Abrimos el archivo type.kt y creamos la FontFamily y la agregamos a nuestra Typography.



# 4. Fuentes

- Aplicamos los cambios en nuestra función SplashScreen().

```
25          Text(text = "Bienvenido",
26              style = MaterialTheme.typography.titleLarge)
27          OutlinedButton(onClick = { Log.d( tag = "SplashScreen",  msg = "Haga click en continuar")
28
29          }) {
30              Text(text = "Continuar",
31                  style = MaterialTheme.typography.bodyLarge)
32          }
```

# 5. Colores

- Material designer cuenta con una página web con información de todos los elementos que componen Material Design: https://m3.material.io/
- En el apartado https://material-foundation.github.io/material-theme-builder/ podemos seleccionar un color primario y uno secundario y se adaptan los otros dos colores y crea una paleta de colores y los temas claro y oscuro que podemos exportar a nuestro proyecto.
- Puedes elegir un color y ver complementarios en https://paletton.com/#uid=1000u0kllllaFw0g0qFqFg0w0aF o puedes generar temas aleatorios en https://huemint.com/brand-intersection/.



# 5. Colores

- Genera dos archivos que podemos copiar a Color.kt y Theme.kt.

- En el texto elegimos uno de los colores exportados escribiendo el nombre de su variable:

color = MaterialTheme.colorScheme.onTertiary

```
@Composable
fun SplashScreen(){
    Column (
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ){
        Image(
            painter = painterResource(id = R.drawable.im_grabi_1),
            contentDescription = "imagen")
        Text(text = "Bienvenido",
            style = MaterialTheme.typography.titleLarge,
            color = MaterialTheme.colorScheme.onTertiary
        )
        OutlinedButton(onClick = { Log.d( tag = "SplashScreen",  msg = "Haga click en continuar")

        }) {
            Text(text = "Continuar",
                style = MaterialTheme.typography.bodyLarge)
        }
    }
}
```

# 6. SplashScreen

- Queremos que al comenzar la aplicación, nos muestre la splashScreen que muestra un botón que nos permitirá ir al menú home, pero además, que al de un tiempo determinado, vaya automáticamente al mismo menú home.

- Para que vaya al menú home, detallado en la MainActivity aplicamos los siguientes cambios:

  - Cabecera: fun SplashScreen(onContinue: () -> Unit){

  - OutlinedButton: Añadimos después del log onContinue()

```
@Composable
fun SplashScreen(onContinue: () -> Unit){
    val context = LocalContext.current

    Column(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Image(
            painter = painterResource(id = R.drawable.im_grabi_1),
            contentDescription = "imagen")
        Text(text = "Bienvenido",
            style = MaterialTheme.typography.titleLarge,
            color = MaterialTheme.colorScheme.onTertiary
        )
        OutlinedButton(onClick = {
            Log.d( tag = "SplashScreen",  msg = "Haga click en continuar")
            onContinue()
        }) {
            Text(text = "Continuar",
                style = MaterialTheme.typography.bodyLarge)
        }
    }
}
```

# 6. SplashScreen

- En la MainActivity, creamos una variable showSplash que nos permite iniciar la función de la clase SplashScreen si es true:
- Un LaunchedEffect que nos permite establecer la duración de la splashScreen:
- Un if-else que una vez visto el splashScreen, nos muestre el contenido de esta actividad:

```
18  class MainActivity : ComponentActivity() {
19      override fun onCreate(savedInstanceState: Bundle?) {
20          super.onCreate(savedInstanceState)
21
22          setContent {
23              var showSplash by remember { mutableStateOf( value = true) }
24
25              LaunchedEffect( key1 = Unit) {
26                  delay( timeMillis = 3000L)
27                  showSplash = false
28              }
29
30              MaterialDesignApplicationTheme {
31                  if (showSplash) {
32                      SplashScreen(onContinue = { showSplash = false })
33                  } else {
34                      Text (text = "Estas en home")
35                  }
36              }
37          }
38      }
39  }
```

# 7. Animaciones

- Vamos a animar la columna para que haga un efecto zoom in-zoom out al comenzar la splashScreen:
- Creamos la variable scale
- Definimos un LaunchEffect para generar el zoom.
- Lo aplicamos el modifier de la columna.

```
28  @Composable
29  fun SplashScreen(onContinue: () -> Unit){
30      val context = LocalContext.current
31      var scale = remember {
32          Animatable( initialValue = 0f)
33      }
34
35      LaunchedEffect( key1 = Unit) {
36          scale.animateTo(
37              targetValue = 0.9f,
38              animationSpec = tween(
39                  durationMillis = 800,
40                  easing = {
41                      OvershootInterpolator( tension = 8f)
42                          .getInterpolation( t = it)
43                  }
44              )
45          )
46      }
47
48      Column(
49          modifier = Modifier.fillMaxSize().scale(scale.value),
50          verticalArrangement = Arrangement.Center,
51          horizontalAlignment = Alignment.CenterHorizontally
52      ) {
```